

Longest Turbulent Subarray DSA

978. Longest Turbulent Subarray

Let's understand this from **zero**, then build the optimal solution.

1. What is the problem asking?

You are given an array.

A **turbulent subarray** is a subarray where the comparison signs **keep changing**:

```
< > < > < ...
```

or

```
> < > < > ...
```

For example:

```
[4, 2, 7, 3]
```

Comparisons:

```
4 > 2
2 < 7
7 > 3
```

So:

```
> < >
```

The signs alternate, so this is turbulent.

Its length is 4.

Example

Given:

```
[9, 4, 2, 10, 7, 8, 8, 1, 9]
```

Consider:

```
[4, 2, 10, 7, 8]
```

Comparisons:

```
4 > 2
2 < 10
10 > 7
7 < 8
```

So:

```
> < > <
```

The signs alternate.

Therefore, its length is:

```
5
```

So the answer is 5.

2. The key observation

Forget about the actual values for a moment.

We only care about whether two neighboring elements are:

```
<
```

```
>
=
```

For example:

```
arr = [9, 4, 2, 10, 7, 8]
```

The comparisons are:

```
9 > 4
4 > 2
2 < 10
10 > 7
7 < 8
```

So:

```
> > < > <
```

A turbulent subarray requires:

```
> <
```

or

```
< >
```

at every step.

Therefore, the moment we see:

```
> >
```

or

```
< <
```

the current turbulent subarray **breaks**.

And if:

```
=
```

appears, it also breaks because equality isn't < or >.

3. Brute-force approach

The most straightforward idea is:

Step 1

Choose every possible starting index.

Step 2

Extend the subarray one element at a time.

Step 3

Check whether the comparisons alternate.

For example:

```
[9, 4, 2, 10]
```

Check:

```
9 > 4
4 > 2
```

Already:

```
> >
```

So it isn't turbulent.

Then try another starting point:

```
[4, 2, 10]
```

We get:

```
4 > 2
2 < 10
```

Good.

Then continue.

Why is this inefficient?

There can be $O(n^2)$ subarrays.

For each subarray, we may spend additional time checking its comparisons.

So a straightforward implementation can become:

```
 $O(n^2)$ 
```

or even worse depending on how we check.

But:

```
n <= 4 * 10^4
```

So we want approximately:

```
 $O(n)$ 
```

4. Let's build the optimal intuition

Instead of checking every subarray, let's scan the array **once**.

Suppose:

```
arr = [9, 4, 2, 10, 7, 8]
```

Comparisons:

```
9 > 4
4 > 2
2 < 10
10 > 7
7 < 8
```

Represent them as:

```
> > < > <
```

Now notice something important.

Whenever the current comparison is different from the previous comparison, the turbulent sequence can continue.

For example:

```
> <
```

Good.

```
< >
```

Good.

But:

```
> >
```

Bad.

```
< <
```

Bad.

5. What should we store?

We can maintain:

```
current = length of current turbulent subarray
answer = maximum length found so far
```

Initially:

```
current = 1
answer = 1
```

Why 1?

A single element is always a valid turbulent subarray because there are no comparisons to violate the rule.

6. Compare adjacent elements

At every index *i*, compare:

```
C++
arr[i]
```

with

```
C++
arr[i - 1]
```

There are three cases.

Case 1: `arr[i] > arr[i-1]`

The current comparison is:

```
<
```

because moving from `arr[i-1]` to `arr[i]`:

```
arr[i-1] < arr[i]
```

If the previous comparison was `>`, we're good.

Case 2: `arr[i] < arr[i-1]`

The current comparison is:

```
>
```

Again, if the previous comparison was `<`, we're good.

Case 3: `arr[i] == arr[i-1]`

We have:

```
=
```

This cannot be part of a turbulent sequence.

So we reset:

```
current = 1
```

7. A very useful trick

We don't actually need to remember the entire previous subarray.

We only need to know the **previous comparison**.

For example:

```
previous = <
```

and current comparison is:

```
>
```

Then they alternate.

So:

```
current++
```

But if:

```
previous = >
```

and current is:

```
>
```

then alternation has broken.

The new turbulent subarray can start from the previous element:

```
current = 2
```

Why 2?

Because the two elements themselves form a valid subarray:

```
[a, b]
```

Any two unequal elements are turbulent.

8. Even simpler implementation

There is a neat way to avoid explicitly storing the previous sign.

Define:

```
C++
prev
```

as:

```
1 -> previous comparison was <
-1 -> previous comparison was >
0 -> no previous comparison
```

For current comparison:

```
C++
curr = 1 if arr[i] > arr[i-1]
curr = -1 if arr[i] < arr[i-1]
curr = 0 if arr[i] == arr[i-1]
```

Then:

If equal

```
C++
current = 1;
```

If signs alternate

```
C++
current++;
```

Otherwise

```
C++
current = 2;
```

And finally:

```
C++
```

```
prev = curr;
```

9. Optimal C++ code

```
C++
class Solution {
public:
    int maxTurbulenceSize(vector<int>& arr) {
        int n = arr.size();

        if (n == 1)
            return 1;

        int ans = 1;
        int current = 1;
        int prev = 0;

        for (int i = 1; i < n; i++) {
            int curr;

            if (arr[i] > arr[i - 1])
                curr = 1;
            else if (arr[i] < arr[i - 1])
                curr = -1;
            else
                curr = 0;

            if (curr == 0) {
                // Equal elements break turbulence
                current = 1;
            }
            else if (prev == 0 || curr != prev) {
                // First comparison OR signs are alternating
                current++;
            }
            else {
                // Same comparison sign appeared twice
                current = 2;
            }

            ans = max(ans, current);

            prev = curr;
        }

        return ans;
    }
};
```

10. Understanding the important part of the code

This is the heart:

```
C++
if (curr == 0) {
    current = 1;
}
```

For example:

```
5, 5
```

Comparison:

```
=
```

So turbulence ends.

Then:

```
C++
else if (prev == 0 || curr != prev) {
    current++;
}
```

There are two situations.

First comparison

```
prev = 0
```

There is no previous comparison, so we can extend:

```
current++
```

Alternating comparison

Suppose:

```
prev = >
curr = <
```

They are different:

```
> <
```

Therefore:

```
C++
current++;
```

Finally:

```
C++
else {
    current = 2;
}
```

Suppose:

```
> >
```

The previous turbulent sequence is broken.

But the last two elements are still a valid turbulent subarray:

```
[a, b]
```

Therefore:

```
current = 2
```

11. Complete dry run

Let's use Example 1:

```
arr = [9, 4, 2, 10, 7, 8, 8, 1, 9]
```

Initial:

```
current = 1
ans = 1
prev = 0
```

We'll track:

i	Pair	Comparison	prev	curr	current	ans
1	9, 4	>	0	-1	2	2
2	4, 2	>	-1	-1	2	2
3	2, 10	<	-1	1	3	3
4	10, 7	>	1	-1	4	4
5	7, 8	<	-1	1	5	5
6	8, 8	=	1	0	1	5
7	8, 1	>	0	-1	2	5
8	1, 9	<	-1	1	3	5

Let's go through it carefully.

i = 1

Elements:

```
9, 4
```

We have:

```
9 > 4
```

So:

```
curr = >
```

There was no previous comparison.

Therefore:

```
current = 2
ans = 2
```

Current turbulent subarray:

```
[9,4]
```

i = 2

Elements:

```
4, 2
```

Comparison:

```
4 > 2
```

So:

```
prev = >
curr = >
```

Same sign.

Therefore turbulence breaks.

We keep only:

```
[4,2]
```

So:

```
current = 2
ans = 2
```

i = 3

Elements:

```
2, 10
```

Comparison:

```
2 < 10
```

Now:

```
> <
```

The signs alternate!

Therefore:

```
current = 3
```

Subarray:

```
[4,2,10]
```

Answer:

```
ans = 3
```

i = 4

Elements:

10,7

Comparison:

$10 > 7$

Previous:

<

Current:

>

So:

< >

Alternating.

Therefore:

current = 4

Subarray:

[4,2,10,7]

i = 5

Elements:

7,8

Comparison:

$7 < 8$

We now have:

> < > <

Perfectly alternating.

Therefore:

current = 5

Subarray:

[4,2,10,7,8]

So:

ans = 5

i = 6

Elements:

8,8

Comparison:

$8 = 8$

Equality breaks turbulence.

Therefore:

current = 1

But:

```
ans = 5
```

still remains.

i = 7

Elements:

```
8,1
```

Comparison:

```
8 > 1
```

This is the first comparison after the reset.

So:

```
current = 2
```

i = 8

Elements:

```
1,9
```

Comparison:

```
1 < 9
```

Now:

```
> <
```

Alternating.

So:

```
current = 3
```

The final answer remains:

```
5
```

12. Example 2

```
arr = [4,8,12,16]
```

Comparisons:

```
4 < 8
8 < 12
12 < 16
```

So:

```
< < <
```

The signs don't alternate.

Process:

```
4,8      -> current = 2
8,12     -> same sign -> current = 2
12,16    -> same sign -> current = 2
```

Therefore:

```
answer = 2
```

13. Example 3

```
arr = [100]
```

There is only one element.

There are no comparisons.

A single element is valid.

Therefore:

```
answer = 1
```

14. Why does `current = 2` make sense?

This is one of the most important parts to understand.

Suppose:

```
[4, 2, 1, 7]
```

Comparisons:

```
4 > 2
2 > 1
1 < 7
```

We get:

```
> > <
```

At:

```
2 > 1
```

the turbulence breaks.

But we don't throw everything away.

The last two elements:

```
[2, 1]
```

are still a valid turbulent subarray.

Therefore:

```
current = 2
```

Then when:

```
1 < 7
```

appears, we can extend:

```
[2, 1, 7]
```

giving:

```
> <
```

So:

```
current = 3
```

This is the key reason we reset to 2, not 1.

15. Complexity

We scan the array exactly once.

Time

```
O(n)
```

because every element is processed once.

Space

```
0(1)
```

because we only store a few variables:

```
ans
current
prev
curr
```

16. Final intuition to remember

For this problem, don't think:

"I need to find every subarray."

Instead think:

"I only need to track whether consecutive comparison signs alternate."

Convert:

```
[9,4,2,10,7,8]
```

into:

```
> > < > <
```

Then:

- Different signs → **continue**
- Same signs → **restart from length 2**
- Equal elements → **restart from length 1**

So the entire problem becomes a simple **one-pass sliding/DP-style scan**:

```
> < > < > < ...
  ↑
alternation = continue
```

That's why we can solve it in:

```
Time : 0(n)
Space : 0(1)
```

Yes! 🙌 There is a **very common and easier DP-style solution** using two variables called **up and down**.

This is actually one of the cleanest ways to solve **Longest Turbulent Subarray**.

1. The basic idea: up and down

Instead of storing the previous comparison, we maintain:

- **up** = length of the turbulent subarray ending at the current position where the **last comparison is <**
- **down** = length of the turbulent subarray ending at the current position where the **last comparison is >**

For example:

```
arr = [4, 2, 7]
```

Comparisons:

```
4 > 2 < 7
```

At 2:

```
4 > 2
```

So:

```
down = 2
```

At 7:

```
2 < 7
```

To have a turbulent sequence, the previous comparison must have been >.

It was!

So:

```
up = down + 1
```

Therefore:

```
up = 3
```

We found:

```
[4, 2, 7]
```

with length 3.

2. Why do we need two variables?

Suppose:

```
4 > 2 < 7 > 3
```

The signs are:

```
> < > <
```

When we see <, we need to know:

How long was the sequence ending with >?

That's exactly what **down** stores.

Similarly, when we see >:

How long was the sequence ending with <?

That's what **up** stores.

So:

```
up   = sequence ending with <
down = sequence ending with >
```

3. The transitions

There are three cases.

Case 1: `arr[i] > arr[i-1]`

We have:

```
arr[i-1] < arr[i]
```

So the current comparison is <.

To continue turbulence, the previous comparison must have been >.

Therefore:

```
C++
up = down + 1;
```

And because the current comparison is not < → **down** no longer represents a valid sequence ending at this position:

```
C++
down = 1;
```

Case 2: `arr[i] < arr[i-1]`

Current comparison is:

>

So we need the previous < sequence.

Therefore:

```
C++
down = up + 1;
```

And:

```
C++
up = 1;
```

Case 3: `arr[i] == arr[i-1]`

Neither < nor >.

So turbulence completely breaks:

```
C++
up = 1;
down = 1;
```

4. C++ code

```
C++
class Solution {
public:
    int maxTurbulenceSize(vector<int>& arr) {
        int up = 1;
        int down = 1;
        int ans = 1;

        for (int i = 1; i < arr.size(); i++) {
            if (arr[i] > arr[i - 1]) {
                up = down + 1;
                down = 1;
            }
            else if (arr[i] < arr[i - 1]) {
                down = up + 1;
                up = 1;
            }
            else {
                up = 1;
                down = 1;
            }
            ans = max(ans, max(up, down));
        }
        return ans;
    }
};
```

5. Let's understand the code intuitively

Initially:

```
up = 1
down = 1
ans = 1
```

Why 1?

A single element is always a valid subarray.

Suppose:

```
arr = [9,4,2,10,7,8]
```

Let's process it.

9 > 4

Current comparison:

```
>
```

So:

```
down = up + 1
      = 1 + 1
      = 2
```

And:

```
up = 1
```

Now:

```
up = 1
down = 2
ans = 2
```

4 > 2

Again:

```
>
```

But the previous comparison was also >.

So we **cannot continue** the same turbulent sequence.

The code does:

```
C++
down = up + 1;
```

Since:

```
up = 1
```

we get:

```
down = 2
```

So:

```
up = 1
down = 2
```

The best subarray is:

```
[4, 2]
```

2 < 10

Now the sign changes:

```
> <
```

Therefore we can continue the turbulent sequence.

We do:

```
C++
up = down + 1;
```

Currently:

```
down = 2
```

Therefore:

```
up = 3
```

So:

```
[4, 2, 10]
```

has length 3.

10 > 7

Again the sign changes:

< >

Therefore:

```
C++
down = up + 1;
```

So:

```
down = 3 + 1
      = 4
```

We now have:

[4, 2, 10, 7]

Length:

4

7 < 8

Again:

> <

So:

```
C++
up = down + 1;
```

Therefore:

```
up = 4 + 1
    = 5
```

Now:

[4, 2, 10, 7, 8]

has length:

5

Answer:

5

6. Full dry run

For:

arr = [9, 4, 2, 10, 7, 8, 8, 1, 9]

i	Comparison	up	down	ans
Start	—	1	1	1
1	9 > 4	1	2	2
2	4 > 2	1	2	2
3	2 < 10	3	1	3
4	10 > 7	1	4	4
5	7 < 8	5	1	5
6	8 = 8	1	1	5
7	8 > 1	1	2	5

i	Comparison	up	down	ans
8	1 < 9	3	1	5

Final:

```
answer = 5
```

7. The most important thing to remember

Think of **up** and **down** like this:

```
up
↓
length when the LAST comparison is <
```

```
down
↓
length when the LAST comparison is >
```

Then the transitions become very logical:

Current <

Previous must be >:

```
C++
up = down + 1;
```

Current >

Previous must be <:

```
C++
down = up + 1;
```

Current =

Everything breaks:

```
C++
up = down = 1;
```

So the core is simply:

```
current <
↓
up = down + 1

current >
↓
down = up + 1

current =
↓
up = down = 1
```

This **up/down** solution is essentially a **2-state DP solution**, and it runs in **O(n) time and O(1) space**.

ChatGPT can make mistakes. Check important info.